SSD_Touch Driver Integration Reference

1. Driver configuration

1.1. Overview

The touch IC drivers currently implemented by the SSD platform are all communicated through I2C. Take the driver example of the public version of Goodix here. Please refer to the driver's suggestion for the actual situation.

1.2. Driver Integration

For the touch driver, please provide the linux version of the driver from the touch IC manufacturer. The touch driver belongs to the input subsystem. After the driver is loaded successfully, the device node /dev/input/eventX will be generated (generally, if no other input subsystem exists, it will generate a device node /dev/input/eventX). is event0)

1. Add the linux driver provided by the IC manufacturer to the kernel/drivers/input/touchscreen/ directory, as shown below, add the driver file link in kernel/drivers/input/touchscreen/Makefile, and use a configuration CONFIG_TOUCHSCREEN_GOODIX control

```
obj-$(CONFIG_TOUCHSCREEN_FUJITSU)    += fujitsu_ts.o
obj-$(CONFIG_TOUCHSCREEN_GOODIX)     += goodix.o
obj-$(CONFIG_TOUCHSCREEN_GSLX680)    += gsl_point_id.o
obj-$(CONFIG_TOUCHSCREEN_GSLX680)    += wrt_gslX680.o
```

2. As shown below, add CONFIG_TOUCHSCREEN_GOODIX in kernel/drivers/input/touchscreen/Kconfig to add make menuconfig configuration switch in kernel config

```
config TOUCHSCREEN_GOODIX
    tristate "Goodix I2C touchscreen"
    depends on I2C
    depends on GPIOLIB || COMPILE_TEST
    select FW_LOADER
    select INPUT_EVDEV
    help
      Say Y here if you have the Goodix touchscreen (such as one
      installed in Onda v975w tablets) connected to your
      system. It also supports 5-finger chip models, which can be
      found on ARM tablets, like Wexler TAB7200 and MSI Primo73.

      If unsure, say N.

      To compile this driver as a module, choose M here: the
      module will be called goodix.
```

3. In the project use stage, open the corresponding config through the make menuconfig configuration to ensure that the driver Build in to the kernel can be

---

2. DTS configuration

2.1. DTS configuration

View the hardware schematic diagram, refer to the driver source code, find the *.dtsi file used by the project, and add the corresponding device tree node to configure for the driver to use. The configuration process is as follows:

1. Configure the compatible name of the device tree node according to the name used by the driver
2. i2c slave addr for touch IC
3. The RST/Interrupts pin connecting the touch IC to the main chip

---

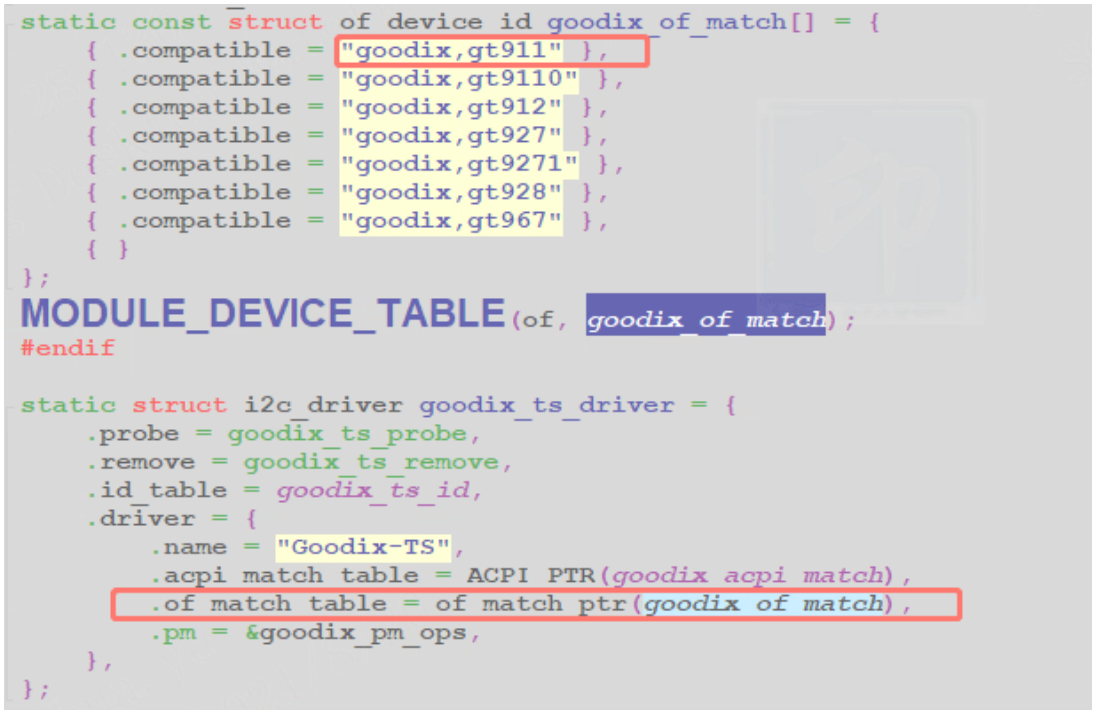
2.2. DTS configuration example

The configuration example of the SSD20X platform is used here. Except that the corresponding *.dts files need to be configured according to the files used, the configuration process is general.

Example use: kernel/arch/arm/boot/dts/infinity2m-ssc011a-s01a-display.dtsi (please add it to the dtsi used by the project during actual use)

1. View the driver source code, find .of_match_table, and create a device node named after this name in dtsi

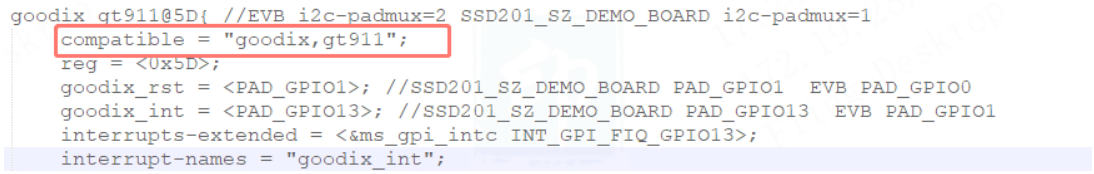
   View the driver source code:



```
static const struct of_device_id goodix_of_match[] = {
    { .compatible = "goodix,gt911" },
    { .compatible = "goodix,gt9110" },
    { .compatible = "goodix,gt912" },
    { .compatible = "goodix,gt927" },
    { .compatible = "goodix,gt9271" },
    { .compatible = "goodix,gt928" },
    { .compatible = "goodix,gt967" },
    { }
};
MODULE_DEVICE_TABLE(of, goodix_of_match);
#endif

static struct i2c_driver goodix_ts_driver = {
    .probe = goodix_ts_probe,
    .remove = goodix_ts_remove,
    .id_table = goodix_ts_id,
    .driver = {
        .name = "Goodix-TS",
        .acpi_match_table = ACPI_PTR(goodix_acpi_match),
        .of_match_table = of_match_ptr(goodix_of_match),
        .pm = &goodix_pm_ops,
    },
};
```

   Configured device node



```
goodix_gt911@5D{ //EVB i2c-padmux=2 SSD201_SZ_DEMO_BOARD i2c-padmux=1
    compatible = "goodix,gt911";
    reg = <0x5D>;
    goodix_rst = <PAD_GPIO1>; //SSD201_SZ_DEMO_BOARD PAD_GPIO1  EVB PAD_GPIO0
    goodix_int = <PAD_GPIO13>; //SSD201_SZ_DEMO_BOARD PAD_GPIO13  EVB PAD_GPIO1
    interrupts-extended = <&ms_gpi_intc INT_GPI_FIQ_GPIO13>;
    interrupt-names = "goodix_int";
```

2. According to the I2C slave addr provided by the touch IC manufacturer, configure the reg attribute. Here, the I2C slave addr address of goodix is 0x5D (it needs to be confirmed by the IC manufacturer)

3. The device tree node configures the pins corresponding to the two attributes of goodix_rst/goodix_int according to the hardware connection (the attribute name needs to match the driver PAD_GPIO1/PAD_GPIO13 (it needs to match the actual hardware), and the reg configuration is touch I2C

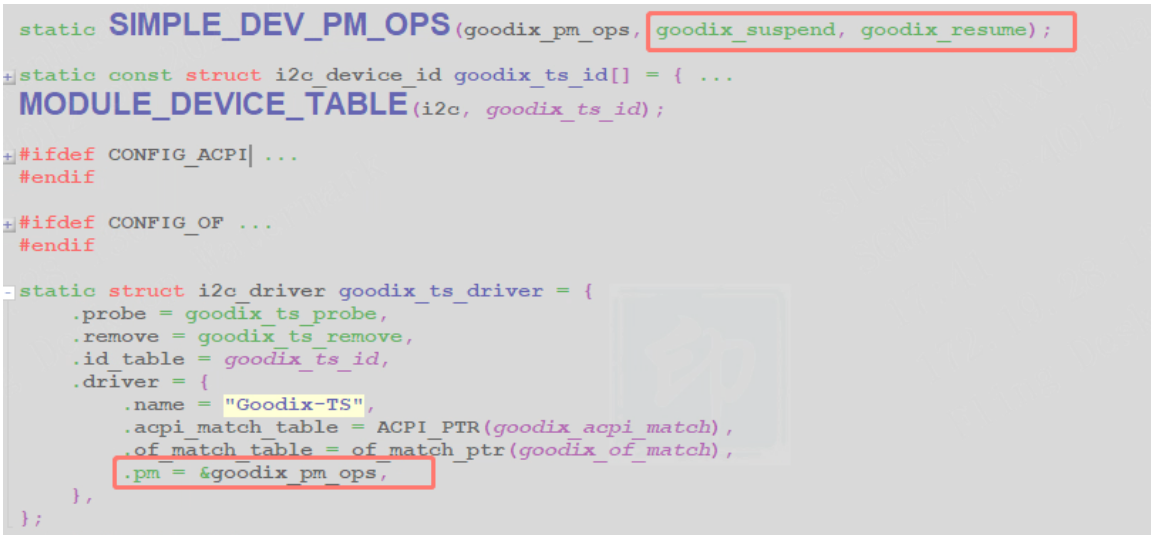
Note: It is best not to repeat the touch devices. If other touch devices are added to the project, please turn off the Goodix driver of the public version Default in the config.

---

3. STR configuration

STR: Suspend To Ram

如需要支持STR，需要跟触控IC的FAE确认触控驱动中已经实现suspend/resume接口，否则STR后触控可能无法恢复导致失效



如需触控唤醒STR功能，则驱动code需在suspend之前先将触控对应中断enable成wake source：

```
static int __maybe_unused goodix_suspend(struct device *dev)
{
    struct i2c_client *client = to_i2c_client(dev);
    struct goodix_ts_data *ts = i2c_get_clientdata(client);
    //int error;

    /* We need gpio pins to suspend/resume */
    if(goodix_ts_fail)
        return 0;
    if (!ts->gpiod_int || !ts->gpiod_rst)
        return 0;
    printk("goodix_suspend start\n");
    goodix_ts_suspending = 1;
    enable_irq_wake(gpio_to_irq(desc_to_gpio(ts->gpiod_int)));
    printk("goodix_suspend end\n");
```

4. 常见FAQ

1. 驱动是否加载成功？

   → 确认对应的触控设备生成的设备节点(eg:/dev/input/eventX)

2. 触控无反应，如何确认通讯是否成功？

   → 查看log有无对应I2C组通讯报错，有报错则可能跟触控IC通讯失败

   → 示波器抓I2C波形，看有无数据交互

3. 如何查看触控版本信息？

   → 使用附件提供Demo:./Evtest /dev/input/eventX 2

4. 触控不准确,会有什么原因？

   → 确保UI绘制的size跟触控的分辨率是匹配的

   → 使用附件提供Demo查看上报的坐标是否准确: ./Evtest /dev/input/eventX 0

   → 使用附件提供Demo模拟输入坐标，触发对应点击: ./Evtest /dev/input/eventX 1 X_ position Y_position

5. Demo

- Demo_Bin

  Evtest [media/Evtest]

- Src_Code

  编译方法： gcc evtest.c -o Evtest

  evtest.c [media/evtest.c]

- 使用说明

  1. 读坐标：

  ./evtest /dev/input/event0 0 //设备节点选择的试 event0 后面一个0是读cmd

  2. 写坐标：

  ./evtest /dev/input/event0 1 90 90 //设备节点选择的试 event0 后面一个1是写cmd，写cmd要带x,y坐标

  3. dump dev info:

  ./evtest /dev/input/event0 2 //设备节点选择的试 event0 后面一个2是Dump Dev相关信息